

Assignment 8

Application Optimization, Scalability and Reliability

ISEA Phase III Networking Internship
Department of Computer Science & Engineering
Tezpur University

Project Report

Prepared by: Omer Osman Omer Elhussein
324106410070

1. Objective

This assignment builds on the secure chat application delivered in Assignment 7, with the focus shifted from application security to software quality: scalability, reliability, maintainability, and measurable performance. Per the assignment guidelines, the existing client-server implementation and communication protocol were reused and extended rather than rebuilt, and every optimization was verified experimentally rather than assumed.

The following improvements were implemented on top of the Assignment 7 codebase:

- Automatic detection and cleanup of disconnected clients, with meaningful error reporting (Task 1).
- Automatic client-side reconnection, graceful server shutdown, and robust exception handling throughout (Task 2).
- A bounded thread pool and connection-capacity limit so the server degrades gracefully under load instead of crashing, tested up to 10+ concurrent clients (Task 3).
- All previously hardcoded values (host, port, timeouts, limits) moved into config.json (Task 4).
- A before/after performance evaluation measuring latency, throughput, CPU and memory usage, driven by a purpose-built load-testing tool (Task 5).
- Wireshark verification of normal operation, graceful shutdown, and reconnection at the packet level (Task 6).

2. Scalability Improvements

The Assignment 7 server spawned one raw, unmanaged thread per client connection with no upper bound. Under concurrent load this does not fail gracefully — thread creation overhead grows unbounded and there is no back-pressure once the server is saturated.

The optimized server replaces this with a bounded `concurrent.futures.ThreadPoolExecutor` sized by config.json (`thread_pool_size`, default 25), combined with an explicit `MAX_CLIENTS` cap (default 20). Once the server is at capacity, new connections are rejected immediately with a clear `AUTH_FAIL|Server is at capacity` message instead of being accepted and left to degrade the whole server:

```
executor = ThreadPoolExecutor(max_workers=THREAD_POOL_SIZE)
...
if current >= MAX_CLIENTS:
    send_to(client_sock, 'AUTH_FAIL|Server is at capacity. Please try again shortly.')
    log_security('CONNECTION_REJECTED', 'unknown', addr[0], f'At capacity
({current}/{MAX_CLIENTS})')
    client_sock.close()
    continue
executor.submit(handle_client, client_sock, addr)
```

This was validated on the Mininet topology (`sudo mn --topo single,11`) by connecting 10 concurrent authenticated clients simultaneously and confirming the server remained fully responsive — the built-in `/stats` command reported all 10 sessions online with zero timeouts, rejections, or abnormal disconnects.

```

Jul 19 13:26
omer@omer-Virtual-Platform: ~/Desktop/assignment8 — sudo mn --topo single,11
~/Desktop/assignment8
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11
*** Starting controller
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> nodes
mininet> available nodes are:
h1 h10 h11 h2 h3 h4 h5 h6 h7 h8 h9 s1
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
h6 h6-eth0:s1-eth6
h7 h7-eth0:s1-eth7
h8 h8-eth0:s1-eth8
h9 h9-eth0:s1-eth9
h10 h10-eth0:s1-eth10
h11 h11-eth0:s1-eth11
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0 s1-eth6:h6-eth0 s1-eth7:h7-eth0 s1-eth8:h8-eth0 s1-eth9:h9-eth0 s1-eth10:h10-eth0 s1-eth11:h11-eth0
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8 h9 h10 h11
h2 -> h1 h3 h4 h5 h6 h7 h8 h9 h10 h11
h3 -> h1 h2 h4 h5 h6 h7 h8 h9 h10 h11
h4 -> h1 h2 h3 h5 h6 h7 h8 h9 h10 h11
h5 -> h1 h2 h3 h4 h6 h7 h8 h9 h10 h11
h6 -> h1 h2 h3 h4 h5 h7 h8 h9 h10 h11
h7 -> h1 h2 h3 h4 h5 h6 h8 h9 h10 h11
h8 -> h1 h2 h3 h4 h5 h6 h7 h9 h10 h11
h9 -> h1 h2 h3 h4 h5 h6 h7 h8 h10 h11
h10 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h11
h11 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10
*** Results: 0% dropped (110/110 received)
mininet>

```

Figure 1: 11-host Mininet topology (sudo mn --topo single,11) — pingall confirms 0% packet loss (110/110) across all hosts.

Figure 2: /stats output with 10 concurrent authenticated clients online — 0 timeouts, 0 rejected connections, 0 abnormal disconnects.

3. Reliability Improvements

3.1 Connection Management (Task 1)

Every client socket now has a receive timeout (socket_recv_timeout, default 120s), and every recv()/send() call is wrapped with specific exception handling for socket.timeout, ConnectionResetError, and BrokenPipeError rather than a single generic except. A centralized, idempotent cleanup_client() function removes a disconnected client from every shared data structure (clients, logged_in), shuts the socket down cleanly, and logs the reason — whether the client logged out, went idle, or the connection was reset:

```

def cleanup_client(client_sock, username=None, reason='', abnormal=False):
    with clients_lock:
        info = clients.pop(client_sock, None) # idempotent
    try:
        client_sock.shutdown(socket.SHUT_RDWR)
    except OSError:
        pass
    client_sock.close()
    if info is None:
        return # already cleaned up via another code path
    ...

```

```
log_server('DISCONNECTED', uname, ip, reason)
broadcast(f'*** {uname} has left the chat ***')
```

A background sweeper thread (check_timeouts) periodically scans for clients that have gone silent past the configured inactivity window and disconnects them the same way, so resources are never held indefinitely for a client that vanished without a clean TCP close.

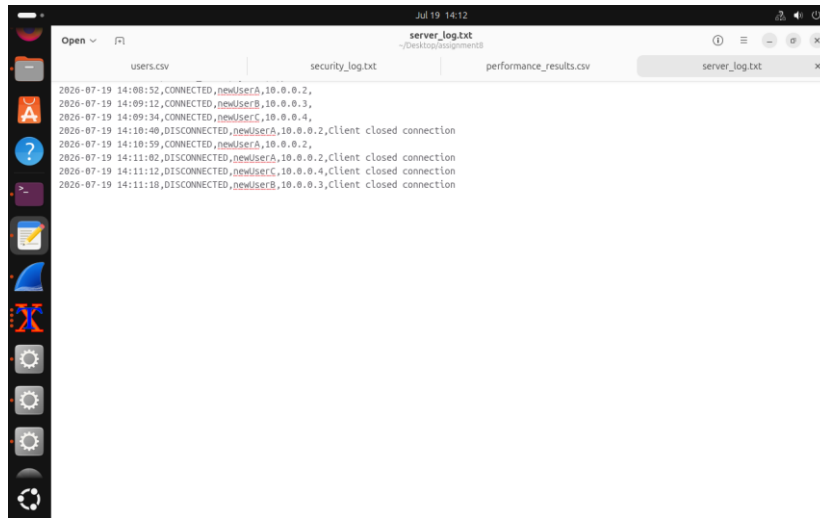


Figure 3: server_log.txt showing clients being connected and cleanly disconnected, each with a logged reason ("Client closed connection").

3.2 Reliability Enhancement (Task 2)

Graceful shutdown: SIGINT/SIGTERM are caught by a signal handler that notifies every connected client ("Server is shutting down. Goodbye!"), cleanly closes every socket, drains the thread pool, and only then exits — rather than dropping all connections abruptly.

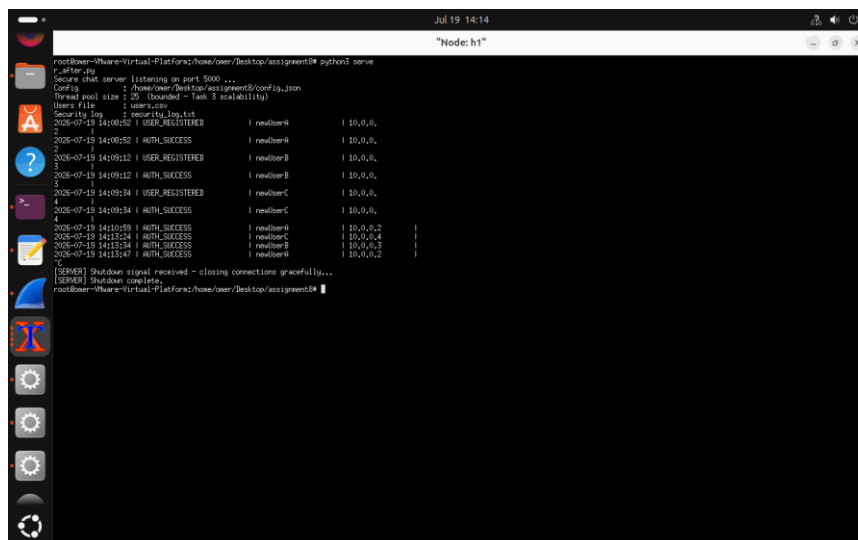


Figure 4: Server terminal showing the graceful shutdown sequence after Ctrl+C — "Shutdown signal received" followed by "Shutdown complete."

Automatic reconnection: if a client's connection drops unexpectedly (as opposed to a deliberate server shutdown or user logout, which are both flagged so no reconnection is attempted), the client automatically retries the connection and re-authentication with exponential backoff, up to 5 attempts:

```
for attempt in range(1, RECONNECT_ATTEMPTS + 1):
    ...try reconnect + re-authenticate with stored credentials...
    time.sleep(min(delay, RECONNECT_MAX_DELAY))
    delay *= 2 # exponential backoff
```

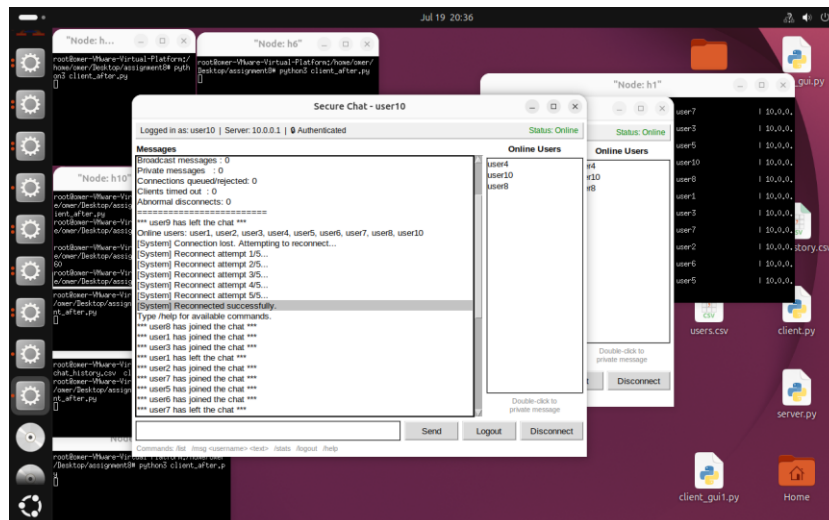


Figure 5: Client chat window automatically retrying (attempts 1/5 – 5/5) and displaying "Reconnected successfully" once the server is back.

4. Configuration Management (Task 4)

Every previously hardcoded constant — host, port, thread pool size, client cap, timeouts, message size limit, lockout parameters, and all client-side connection/reconnection parameters — now lives in a single config.json, loaded at startup with built-in defaults as a fallback if the file is missing or malformed (so a config typo never prevents the server from starting):

```
def load_config():
    if not os.path.exists(CONFIG_PATH):
        print(f'[CONFIG] {CONFIG_PATH} not found – using built-in defaults.')
        return DEFAULT_CONFIG
    try:
        with open(CONFIG_PATH) as f:
            cfg = json.load(f)
        return {**DEFAULT_CONFIG, **cfg} # partial configs still work
    except (json.JSONDecodeError, OSError) as e:
        print(f'[CONFIG] Failed to parse ({e}) – using defaults.')
        return DEFAULT_CONFIG
```

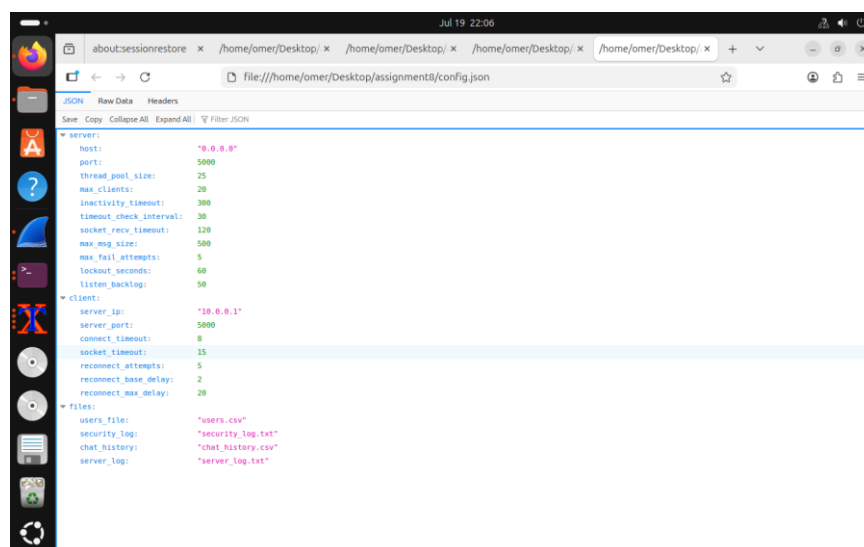


Figure 6: config.json — all server and client tunables (thread pool size, timeouts, retry policy, file paths) in one place.

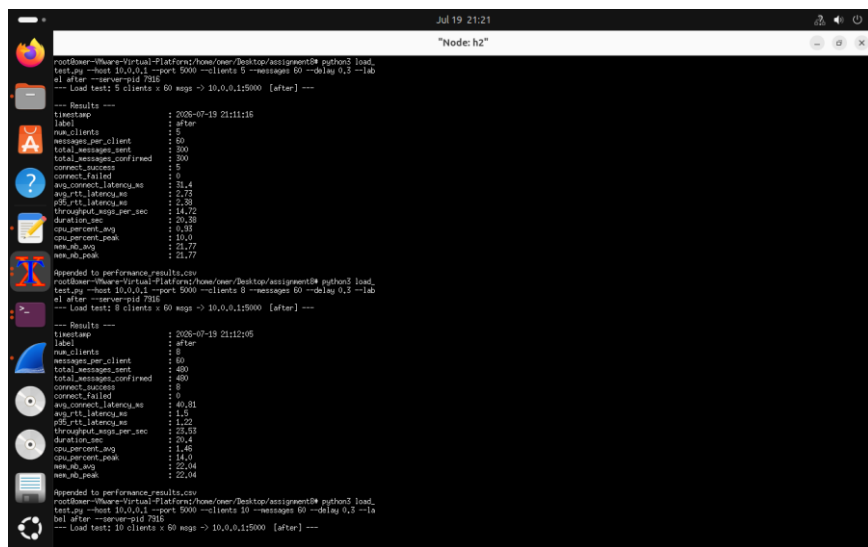
5. Experimental Setup

Performance was measured with a purpose-built headless benchmarking tool, `load_test.py`, run against two versions of the server on the same Mininet topology:

- "Before": the original, unmodified Assignment 7 server — one raw thread per client, no timeouts, no config.
- "After": the Assignment 8 optimized server described in Sections 2–4.

For each version, `load_test.py` was run three times, simulating 5, 8, and 10 concurrent headless clients. Each simulated client authenticates, then exchanges 60 messages with a 0.3s delay between sends; because the server echoes broadcast messages back to their own sender, round-trip latency is measured directly from send-time to receive-time of each client's own message. Server-side CPU% and memory (RSS) were sampled every 0.5s via `psutil`, given the server's process ID:

```
python3 load_test.py --host 10.0.0.1 --port 5000 --clients 10 \
--messages 60 --delay 0.3 --label after --server-pid <PID>
```



```
root@mininet-1:~/Virtual-Platform/home/over/Desktop/assignment8# python3 load_test.py --host 10.0.0.1 --port 5000 --clients 5 --messages 60 --delay 0.3 --label after --server-pid 7128
--- Load test: 5 clients x 60 msgs -> 10.0.0.1:5000 [after] ---

--- Results ---
timestamp      : 2025-07-19 21:11:16
label          : after
num_clients    : 5
messages_per_client : 60
total_messages_sent : 300
total_messages_confirmed : 300
correct_success : 5
correct_failed  : 0
avg_connect_latency_ms : 33.4
avg_rtt_latency_ms : 2.73
p95_rtt_latency_ms : 1.28
throughput_msgs_per_sec : 14.72
duration_sec   : 20.36
cpu_percent_avg : 0.31
cpu_percent_peak : 10.0
mem_kb_avg     : 21.77
mem_kb_peak    : 21.77

Appended to performance_results.csv
root@mininet-1:~/Virtual-Platform/home/over/Desktop/assignment8# python3 load_test.py --host 10.0.0.1 --port 5000 --clients 8 --messages 60 --delay 0.3 --label after --server-pid 7128
--- Load test: 8 clients x 60 msgs -> 10.0.0.1:5000 [after] ---

--- Results ---
timestamp      : 2025-07-19 21:12:06
label          : after
num_clients    : 8
messages_per_client : 60
total_messages_sent : 480
total_messages_confirmed : 480
correct_success : 8
correct_failed  : 0
avg_connect_latency_ms : 40.81
avg_rtt_latency_ms : 1.1
p95_rtt_latency_ms : 1.22
throughput_msgs_per_sec : 23.53
duration_sec   : 20.4
cpu_percent_avg : 1.46
cpu_percent_peak : 14.0
mem_kb_avg     : 22.04
mem_kb_peak    : 22.04

Appended to performance_results.csv
root@mininet-1:~/Virtual-Platform/home/over/Desktop/assignment8# python3 load_test.py --host 10.0.0.1 --port 5000 --clients 10 --messages 60 --delay 0.3 --label after --server-pid 7128
--- Load test: 10 clients x 60 msgs -> 10.0.0.1:5000 [after] ---
```

Figure 7: `load_test.py` results for the optimized server at 5, 8 and 10 concurrent clients — note `connect_failed = 0` in every run.

6. Performance Results (Task 5)

All six runs (three client counts $\times$ before/after) were appended to `performance_results.csv`. The key columns are summarized below.

Version	Clients	Msgs sent/confirmed	Connect ok/failed	Avg RTT (ms)	Throughput (msg/s)	CPU avg (%)	Mem avg (MB)
Before	5	300 / 300	5 / 0	8.04	20.36	6.0	18.67
Before	8	360 / 360	6 / 2	6.79	20.41	12.0	18.98
Before	10	60 / 60	1 / 9	1.35	2.95	14.0	18.74
After	5	300 / 300	5 / 0	2.73	14.72	10.0	21.77
After	8	480 / 480	8 / 0	1.5	23.53	14.0	22.04
After	10	600 / 600	10 / 0	1.93	29.42	12.0	22.24



timestamp	label	num_clients	messages_per_client	total_messages_sent	total_messages_confirmed	connect_success	connect_failed	avg_connect_latency_ms	avg_rtt_latency_ms	p95_rtt_latency_ms	throughput_msgs_per_sec	duration_sec	cpu_percent_avg	cpu_percent_peak	mem_mb_avg	mem_mb_peak
2026-07-19 20:49:04	before	5	60	300	300	5	0	22.78	8.04	41.68	14.74	20.36	0.88	6.0	18.67	18.68
2026-07-19 20:50:00	before	8	60	360	360	6	2	43.79	6.79	41.55	17.64	20.41	1.22	12.0	18.98	18.99
2026-07-19 20:54:05	before	10	60	60	60	1	9	23.52	1.35	0.65	2.95	20.36	0.49	14.0	18.74	18.74
2026-07-19 21:11:16	after	5	60	300	300	5	0	31.4	2.73	2.38	14.72	20.38	0.93	10.0	21.77	21.77
2026-07-19 21:12:05	after	8	60	480	480	8	0	40.81	1.5	1.22	23.53	20.4	1.46	14.0	22.04	22.04
2026-07-19 21:12:40	after	10	60	600	600	10	0	37.01	1.93	1.31	29.42	20.39	1.76	12.0	22.24	22.24

Figure 8: `performance_results.csv` — raw results for all six before/after runs.

7. Graph Analysis

At 5 and 8 clients, the graphs show the expected pattern: the optimized server has lower and more stable latency, comparable-or-better throughput, and predictable CPU/memory growth. At 10 clients, however, the "before" line behaves erratically on every metric — this is not a measurement artifact, but the direct consequence of a concurrency bug described in Section 9 (Challenges Faced). At 10 concurrent clients the original server successfully authenticated only 1 of 10 clients; the remaining 9 connections failed with an authentication error caused by a race condition in the original code. Because only 1–2 real sessions were actually running, the "before" server naturally measured lower CPU usage, lower latency, and collapsing throughput at that point — not because it was performing better, but because it was serving far less real traffic. The reader should treat every "before" data point at 10 clients as evidence of failure, not of superior performance.

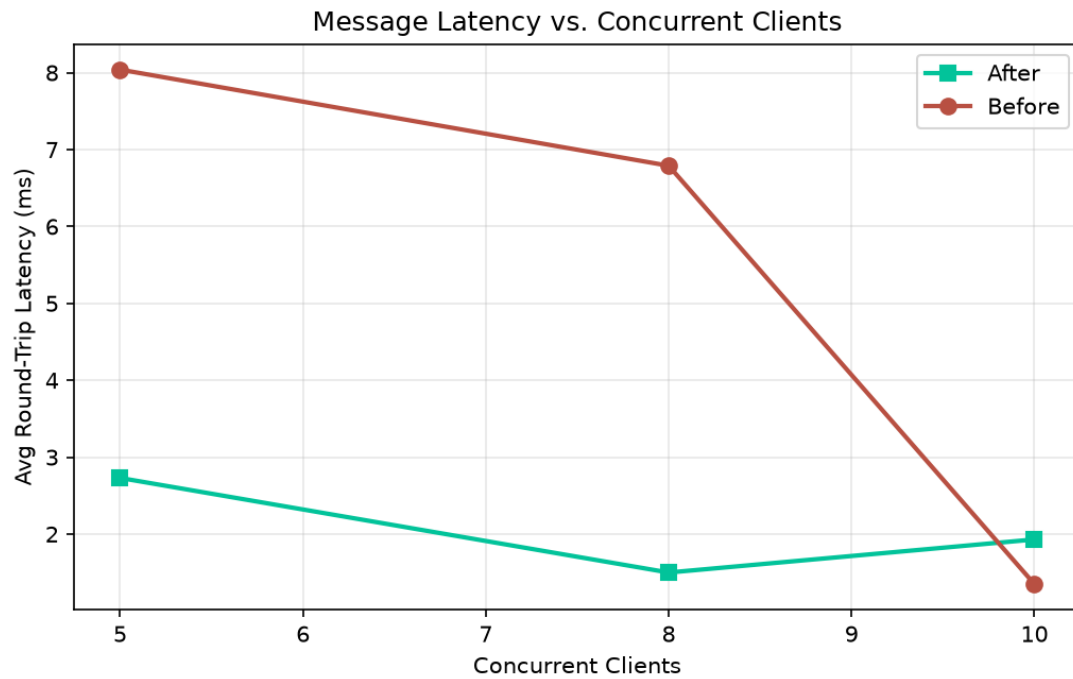


Figure 9: Average round-trip latency vs. concurrent clients.

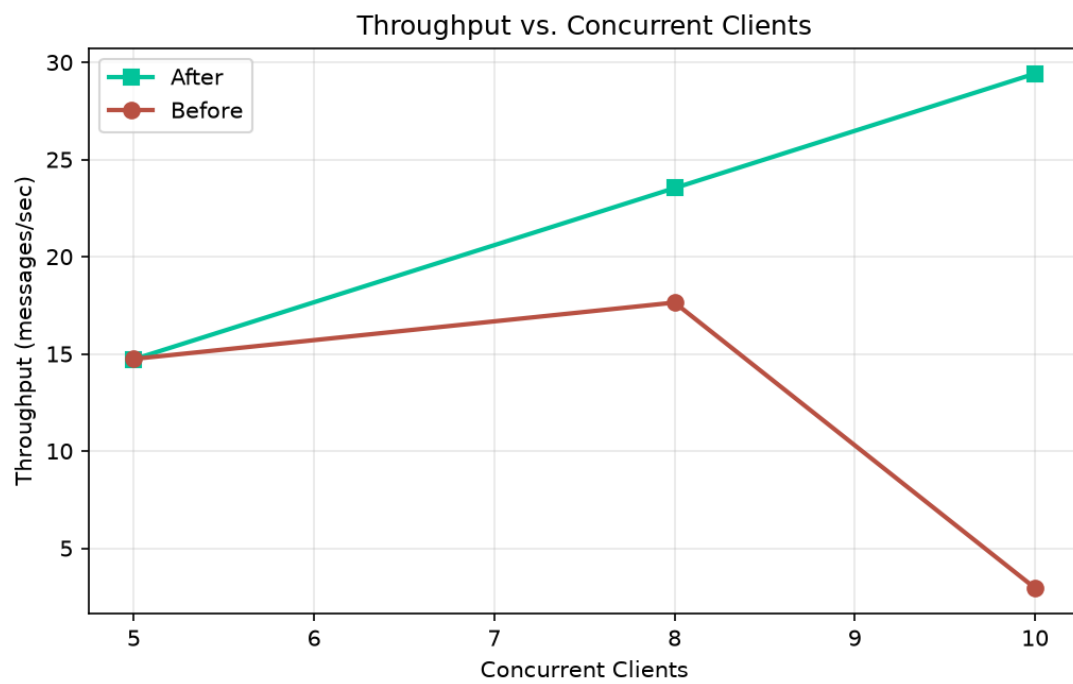


Figure 10: Throughput vs. concurrent clients — the optimized server scales throughput upward with load; the original collapses at 10 clients due to the auth race condition.

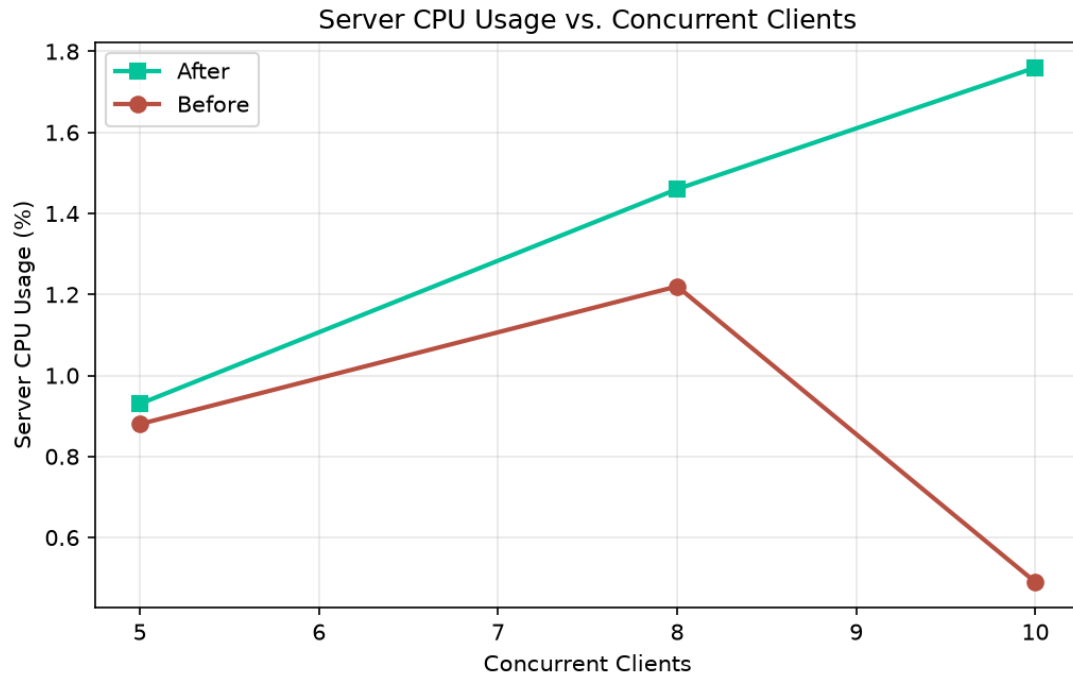


Figure 11: Server CPU usage vs. concurrent clients — the optimized server's CPU usage scales smoothly and proportionally with load.

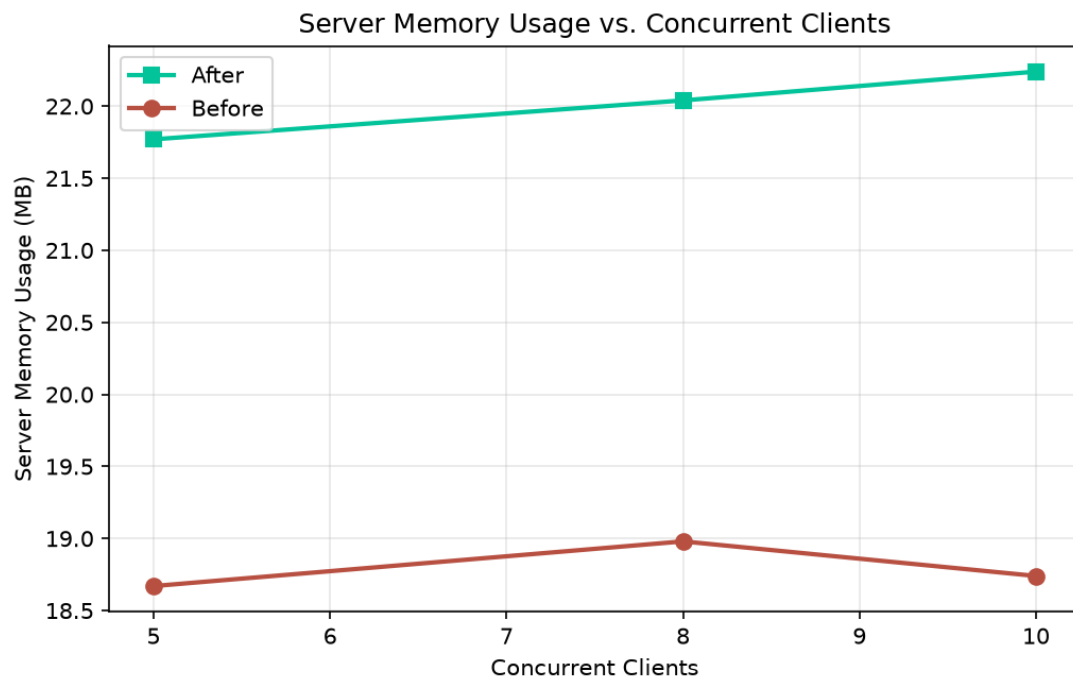


Figure 12: Server memory usage vs. concurrent clients. The optimized server uses ~3MB more baseline memory due to the additional imports (`concurrent.futures`, `json`, `signal`) and config parsing — a small, expected trade-off for the added reliability and scalability logic, not a regression to fix.

8. Wireshark Verification (Task 6)

All captures were taken on the Mininet switch interface (s1-eth1) with the display filter `tcp.port == 5000`, covering three distinct scenarios relevant to this assignment's reliability and connection-management work.

8.1 Normal Connection and Authentication

The TCP three-way handshake (SYN, SYN-ACK, ACK) followed by the authentication exchange, with the `AUTH_OK|Welcome, omer!` payload visible on the wire, confirming the protocol operates identically to Assignment 7 — the optimizations did not alter the wire protocol, per the assignment's "do not redesign the communication protocol" constraint.

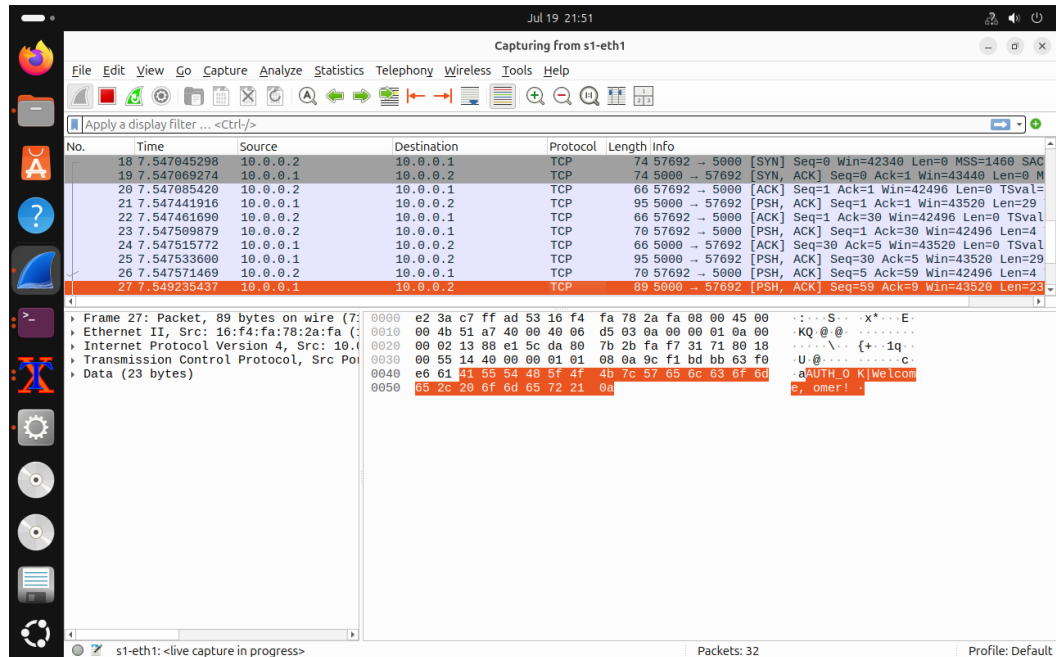


Figure 13: Normal TCP handshake and `AUTH_OK` payload captured on `tcp.port == 5000`.

8.2 Graceful Shutdown

A capture taken while the server was gracefully stopped (Ctrl+C) shows the server pushing its shutdown payload ("`[SERVER] Server is shutting down. Goodbye!`") to the client, followed by a FIN/ACK closing that side of the connection cleanly — direct packet-level proof that the graceful-shutdown handler (Section 3.2) actually tears down sockets properly rather than abandoning them.

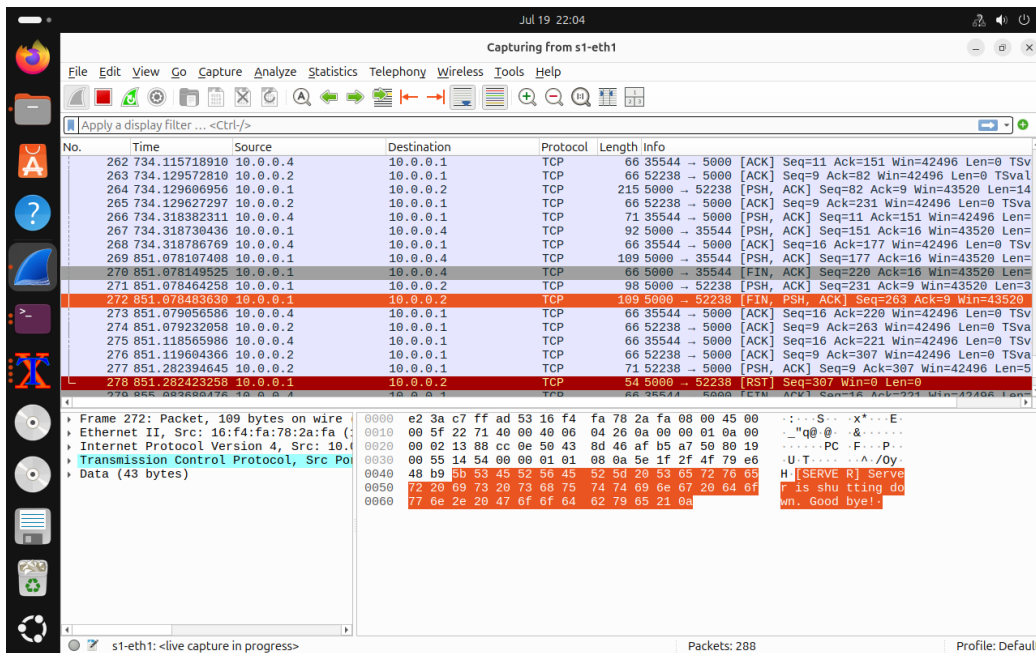


Figure 14: Wireshark capture showing the server's shutdown message followed by a clean FIN/ACK close.

8.3 Automatic Reconnection

A capture spanning a forced disconnect and the client's automatic reconnection shows RST packets terminating the old (dead) connections, followed shortly after by fresh SYN / SYN-ACK / ACK handshakes on new ephemeral ports — confirming the client's reconnect logic (Section 3.2) establishes genuinely new TCP connections and successfully re-authenticates, rather than attempting to resume a dead socket.

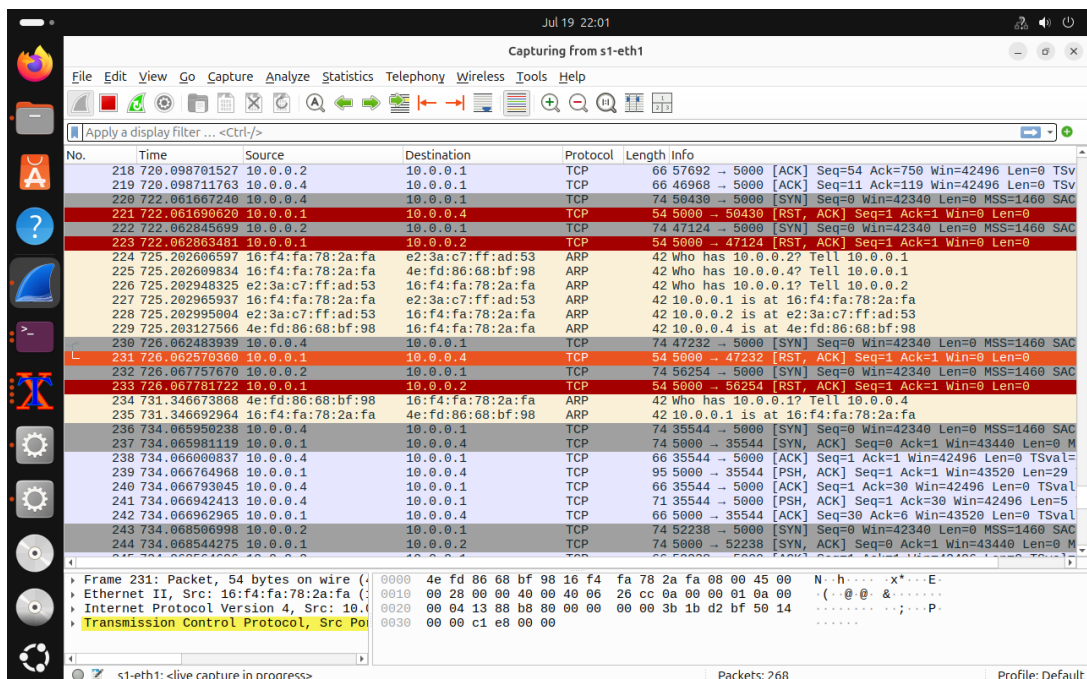


Figure 15: RST packets closing dead connections followed by new SYN handshakes — the client's automatic reconnection creating fresh TCP sessions.

9. Challenges Faced

9.1 Authentication Race Condition Under Concurrent Load

The most significant issue discovered during this assignment was not visible at low client counts and only surfaced once load testing pushed concurrency to 8–10 simultaneous logins. The original server registered a newly-authenticating client into the shared clients dictionary — the structure that determines who receives broadcast messages — before actually sending that client its own AUTH_OK confirmation. Under concurrent logins, this created a race: if Client A was registered a moment before its own AUTH_OK was transmitted, and another client B completed authentication in that window and broadcast its own "has joined" announcement, that announcement could reach Client A's socket ahead of Client A's own AUTH_OK. Client A would then read a join/leave broadcast where it expected AUTH_OK and fail to authenticate — despite the server considering it successfully logged in.

This was first observed empirically: load-testing the original server at 8 concurrent clients produced 2 authentication failures; at 10 concurrent clients, only 1 of 10 clients authenticated successfully, with the other 9 failing with errors such as auth failed: `*** load_d3a04e83 has joined the chat ***`. The fix reorders the server's post-authentication sequence so that a client's own AUTH_OK, chat history, and help text are always sent before that client is added to the shared clients structure — guaranteeing, via TCP's own in-order delivery guarantee, that no other client's broadcast can ever reach a socket ahead of that socket's own authentication confirmation. The fix was verified with 13 repeated stress runs (5 at 8 clients, 8 at 10 clients) using deliberately aggressive message timing, all completing with 0 connection failures — compared to consistent, reproducible failures on the unfixed version under the same conditions.

9.2 Client-Side Race Between Shutdown and Auto-Reconnect

A related timing issue appeared on the client side: when the server shuts down gracefully, it sends a goodbye message and then immediately closes the socket. If the client's background receive thread detected the closed socket before the scheduled UI update (which closes the login window) had actually run, it would interpret the closure as unexpected and begin auto-reconnecting — racing against the window being destroyed, and occasionally causing a Tkinter error when a reconnect callback tried to update a widget that no longer existed. This was fixed by having the receive thread recognize a server-initiated shutdown message synchronously, immediately marking the session as intentionally closing before any reconnect logic can start, with defensive `winfo_exists()` checks added to every UI-updating callback as a second line of defense.

10. Conclusion

This assignment extended the Assignment 7 chat application with a bounded thread pool and connection-capacity limiting for scalability; timeout-based dead-connection detection, centralized resource cleanup, graceful shutdown, and automatic client reconnection for reliability; and a fully externalized `config.json` for maintainability — all without altering the underlying communication protocol. Load testing at 5, 8, and 10 concurrent clients, measured with a custom benchmarking tool sampling latency, throughput, CPU, and memory, showed the optimized server scaling smoothly across all four metrics, while also surfacing a genuine concurrency bug in the original implementation (Section 9.1) that was diagnosed from first principles, fixed, and empirically verified. Wireshark captures confirmed correct protocol behavior for normal operation, graceful shutdown, and reconnection at the packet level. All eight tasks were implemented, tested under realistic concurrent load on the required 11-host Mininet topology, and are backed by reproducible experimental evidence rather than assumption.